

Leva.Framework.Notifications

`Leva.Framework.Notifications` is the provider-neutral notification contract layer of ArchFramework. It defines notifications, recipients, channels, notification entries, gateways, stores, and the application-facing notification service used by provider libraries and applications.

Notifications exists so applications can send user-facing or system-facing messages without coupling workflows, states, services, or hosts to a specific delivery technology. It contains compact contracts, small notification values, and framework-owned orchestration. It does not contain ASP.NET integration, email delivery, push delivery, SignalR/browser updates, queues, retries, templates, user preferences, read/unread inbox state, provider credentials, or application-specific notification categories.

`Leva.Framework.Notifications` depends on `Leva.Framework.Core` so notification operations use the same `Result`, `Result<T>`, `Error`, and `IClock` values as the rest of the framework. Core must not depend on Notifications, and Engine should not depend on Notifications directly; hosts and application access objects should connect notifications explicitly at the application boundary.

```
Leva.Framework.Notifications
    -> Leva.Framework.Core
```

Project overview

Notifications is intentionally provider-neutral. A `Notification` represents one message created for one recipient and one delivery channel. A `NotificationEntry` records the stored outcome of a send operation. The entry vocabulary is consistent with other framework history/current-state values such as runtime entries, command entries, alarm entries, audit entries, and storage entries.

Sending is separated from orchestration. `INotificationGateway` is the provider boundary that sends a notification through a concrete mechanism and returns only `Result`. It does not create stored entries, choose channels, retry, queue work, or simulate failures. `NotificationService` creates the notification, calls the gateway, creates the sent, failed, or cancelled `NotificationEntry`, stores the entry through `INotificationStore`, and returns the entry to the caller.

The core library is not tied to Storage. `INotificationStore` is only the notification-specific boundary for storing entries during the current provider workflow. Memory can keep entries in process, ASP.NET can expose notifications through a web host, email providers can send messages through SMTP or external services, and push providers can send browser/mobile push notifications without changing this library.

Files and classes

Notification model

`NotificationId` - Stable identifier for one notification. `NotificationRecipient` - Identifies who should receive a notification, with a stable ID and optional channel-dependent address and display name. `NotificationChannel` - Open value object that identifies a delivery channel. Provider libraries and applications create channel values such as `email`, `in-app`, `signalr`, or `push` without changing this project. `Notification` - Represents one notification created for one recipient and one channel. It contains the subject, body, and creation time directly to keep the API compact. `NotificationStatus` - Describes the stored outcome of a send operation: sent, failed, or cancelled. `NotificationEntry` - Represents one stored notification entry with the notification, status, completion time, and optional structured error.

Notification contracts

`INotificationGateway` - Sends notifications through a concrete delivery mechanism and returns only operation success or failure. `INotificationStore` - Saves, loads, lists, and deletes notification entries without exposing provider-specific storage details.

Notification service and errors

`NotificationService` - Creates notifications with `IClock`, sends them through `INotificationGateway`, creates notification entries, stores them, and returns the resulting entry. `NotificationErrors` - Creates common structured errors for failed operations, unavailable gateways, unsupported channels, and cancelled sends.